

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Отчет

по лабораторной работе № 2

по дисциплине «Тестирование программного обеспечения»

Вариант: 4567

Выполнил: Молчанов А.Д., группа Р3319
Преподаватель: Тюрин И.Н.

Санкт-Петербург, 2026 г.

Текст задания

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

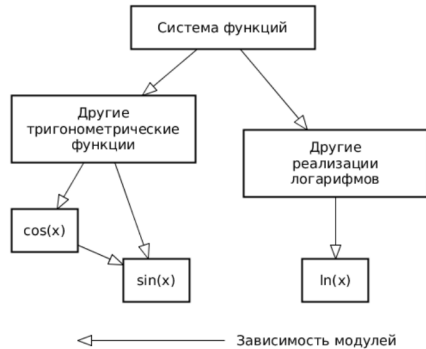
Введите вариант:

$$\begin{cases} \left(\left(\left(\left(\sin(x) + \cos(x) \right) - \cos(x) \right)^3 \cdot \sec(x) \right) + \left(\frac{\cos(x)}{\sin(x) - \cos(x)} \right) \right) & \text{if } x \leq 0 \\ \left(\left(\left(\left(\frac{\log_{10}(x)}{\log_2(x)} \right) + \log_2(x) \right) - \left(\log_{10}(x) \cdot \left(\log_2(x) + \log_{10}(x) \right) \right) \right)^3 + \log_3(x) \right) & \text{if } x > 0 \end{cases}$$

$x \leq 0 : (((((\sin(x) + \cos(x)) - \cos(x)) ^ 3) * \sec(x)) + (\cos(x) / (\sin(x) - \cos(x))))$
 $x > 0 : (((((\log_{10}(x) / \log_2(x)) + \log_2(x)) - (\log_{10}(x) * (\log_2(x) + \log_{10}(x)))) ^ 3) + \log_3(x))$

Правила выполнения работы:

- Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).
- Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведен для базовой тригонометрической функции $\sin(x)$):

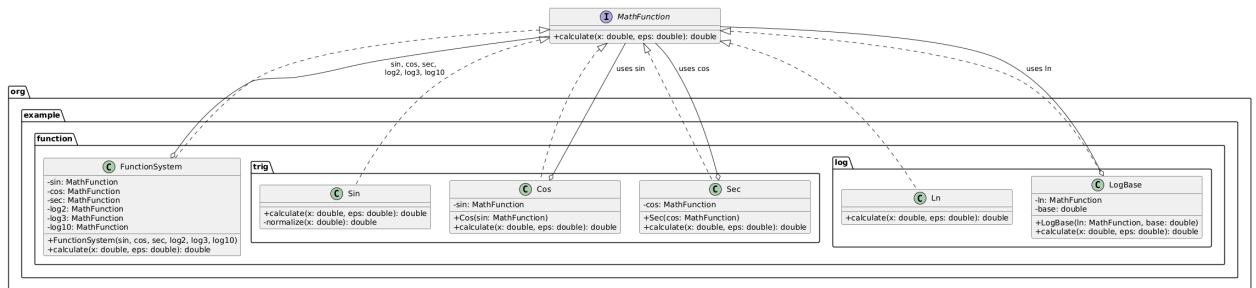


- Обе "базовые" функции (в примере выше - $\sin(x)$ и $\ln(x)$) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.
- Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.
- Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

Порядок выполнения работы:

- Разработать приложение, руководствуясь приведёнными выше правилами.
- С помощью JUNIT5 разработать тестовое покрытие системы функций, проведя анализ эквивалентности и учитывая особенности системы функций. Для анализа особенностей системы функций и составляющих ее частей можно использовать сайт <https://www.wolframalpha.com/>.
- Собрать приложение, состоящее из заглушек. Провести интеграцию приложения по 1 модулю, с обоснованием стратегии интеграции, проведением интеграционных тестов и контролем тестового покрытия системы функций.

UML-диаграмма классов



Описание тестового покрытия и стратегия тестирования

В ходе выполнения лабораторной работы была применена стратегия интеграционного тестирования **«снизу вверх» (Bottom-Up)**.

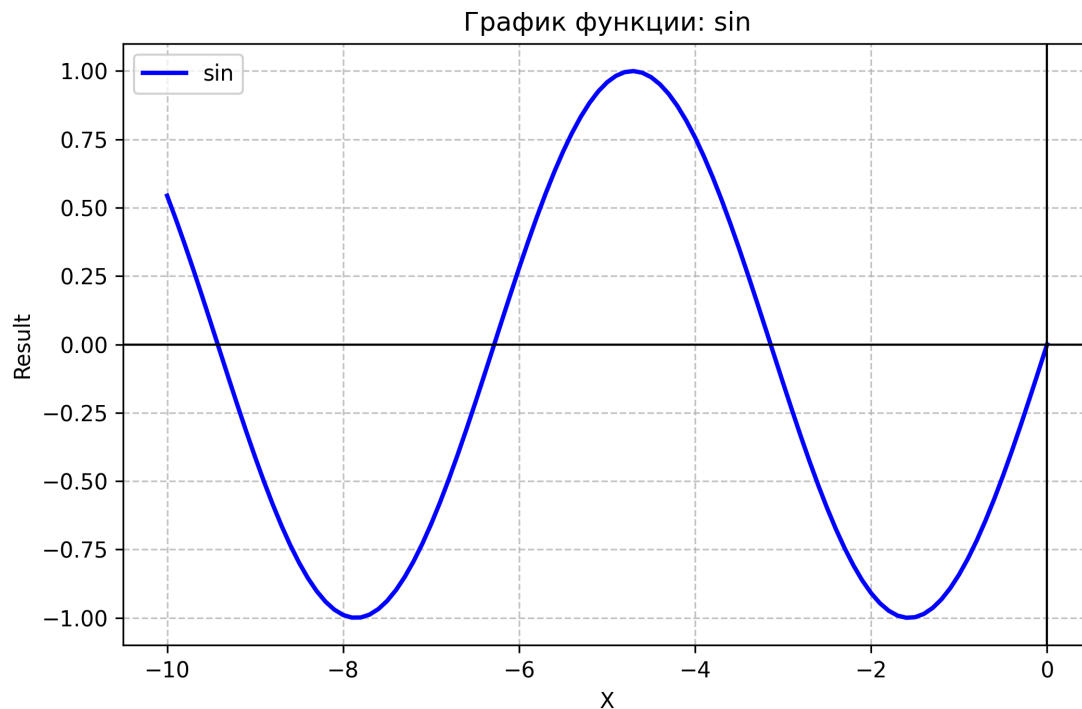
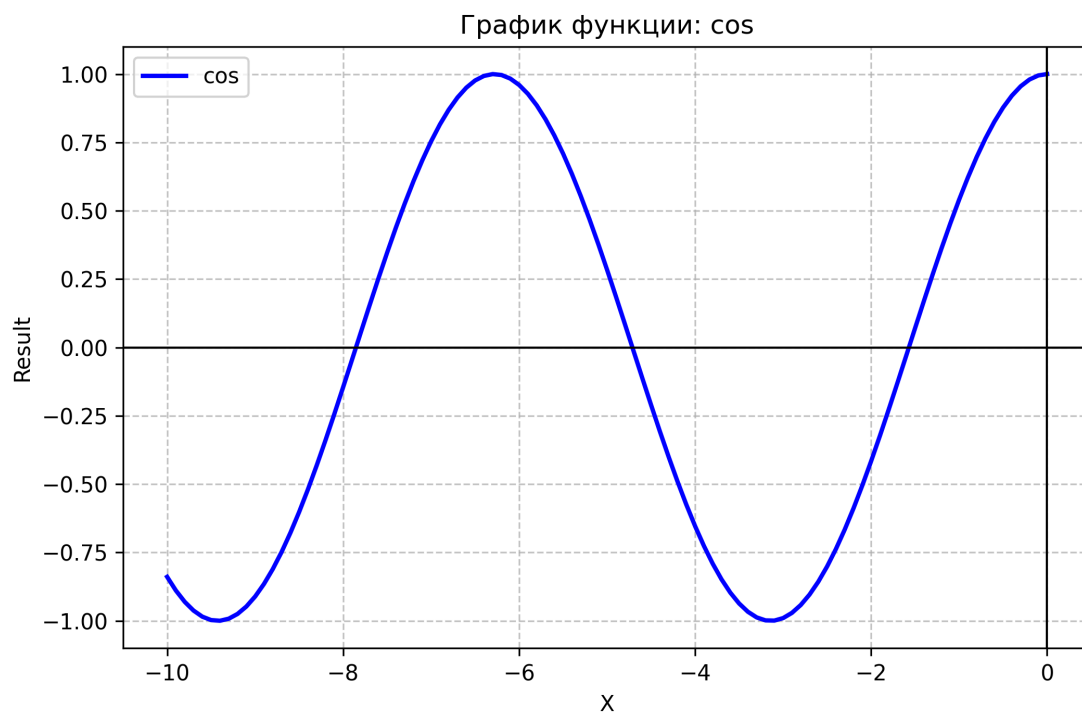
Поскольку наша архитектура строго разделена на слои (базовые $\sin/\ln \rightarrow$ производные $\cos/\sec$ /логарифмы $\rightarrow$ итоговая система `FunctionSystem`), тестирование итогового класса проводилось с использованием mock-объектов (Mockito). Это позволило изолировать верхнюю формулу от погрешностей вычисления рядов Тейлора на нижних уровнях и проверить исключительно логику сборки самой функции.

Набор тестовых данных для `FunctionSystem` подбирался по методу **эквивалентного разбиения и анализа граничных значений**. Были выделены два основных класса эквивалентности, соответствующих ветвлению в формуле:

1. **$x \leq 0$ (тригонометрическая ветвь)**. Для проверки работы сложения, кубической степени и деления тестировались значения из отрицательной области (например, -0.5 и -1.0), а также взята точка -3.14159 ($-\pi$), где синус обнуляется. В нуле функция не имеет неопределенностей, но попадает именно в эту ветвь.
2. **$x > 0$ (логарифмическая ветвь)**. Тестировались целые числа (2.0 , 10.0) и дробные значения больше нуля (0.5). Точка $x = 0$ была исключена, так как логарифмы не существуют для неположительных чисел, и наша исходная система это успешно перехватывает первым правилом $x \leq 0$.

Такого тестового набора (@CsvSource) достаточно для полного покрытия всех математических операций внутри итогового класса без дублирования однотипных проверок. Все сравнения результатов проводились с допустимой дельтой погрешности $\varepsilon = 0.001$.

Графики тригонометрических и логарифмических функций



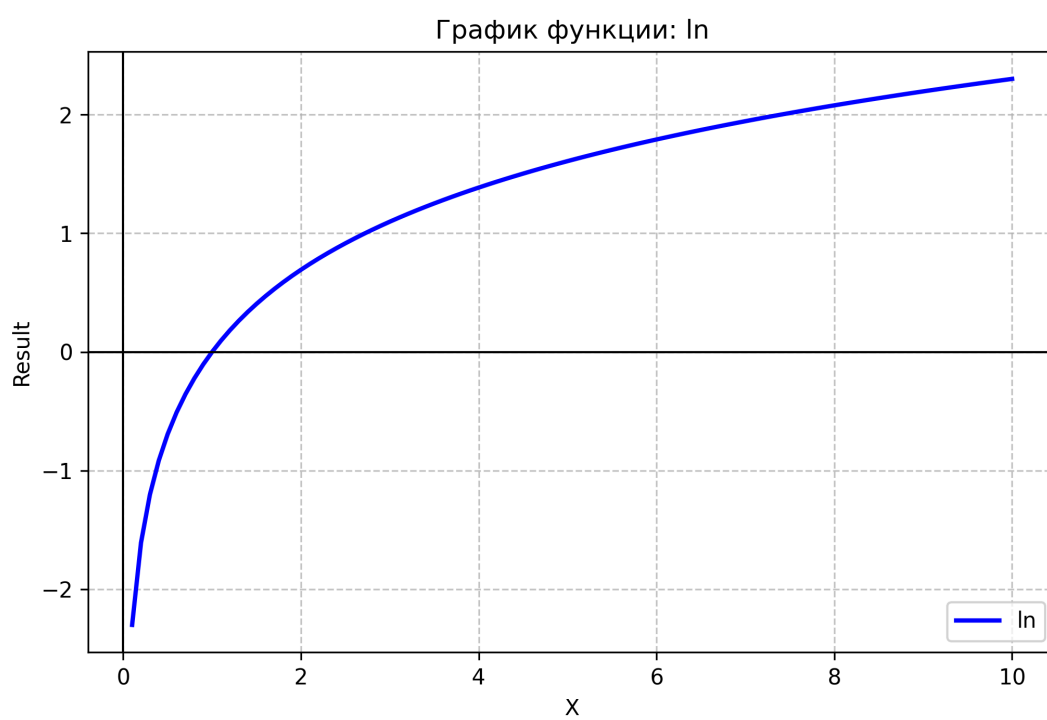
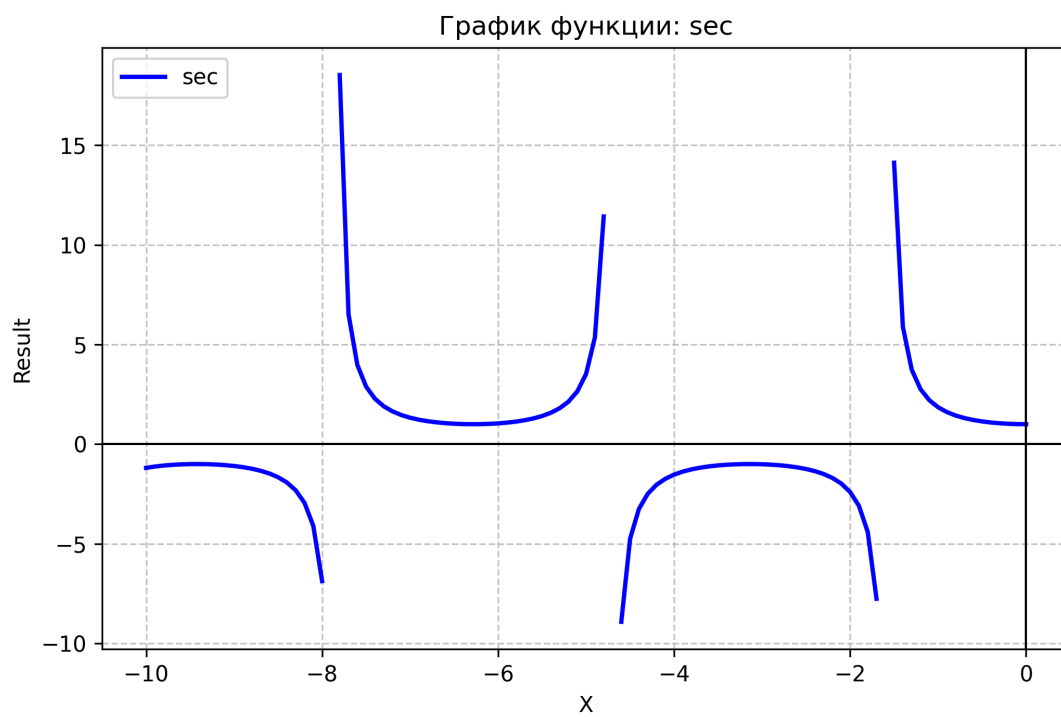


График функции: log2

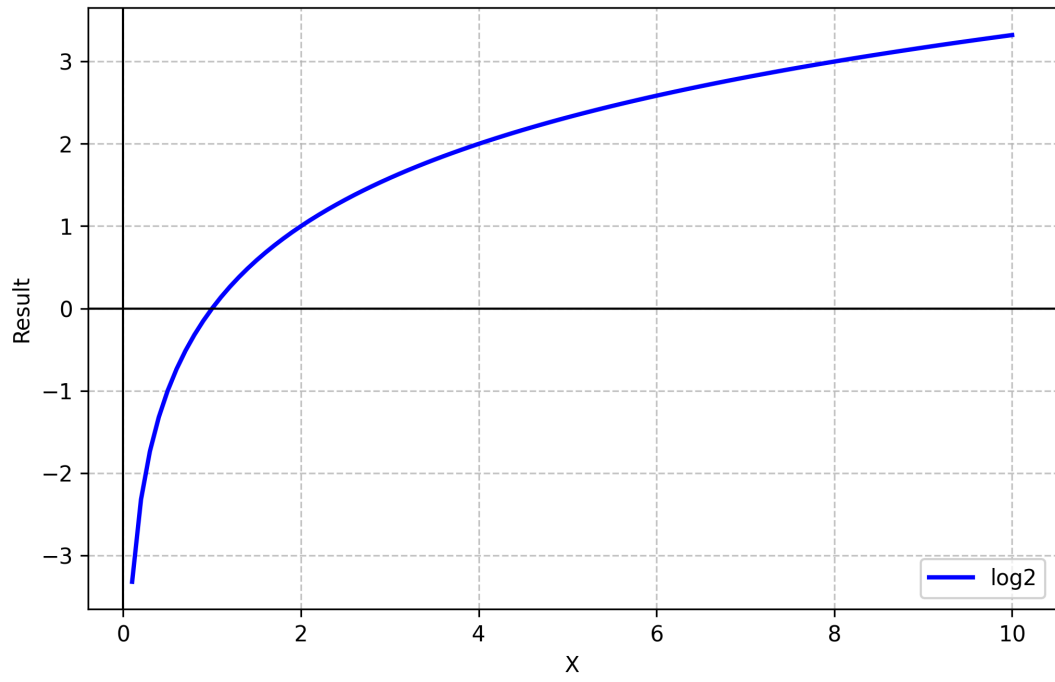


График функции: log10

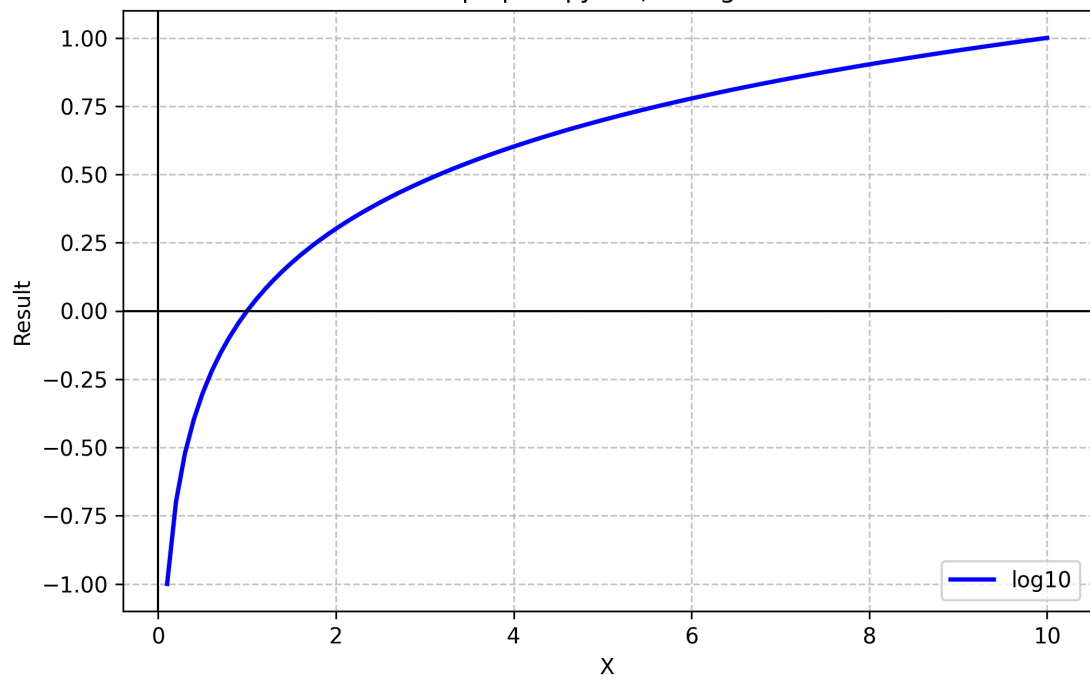
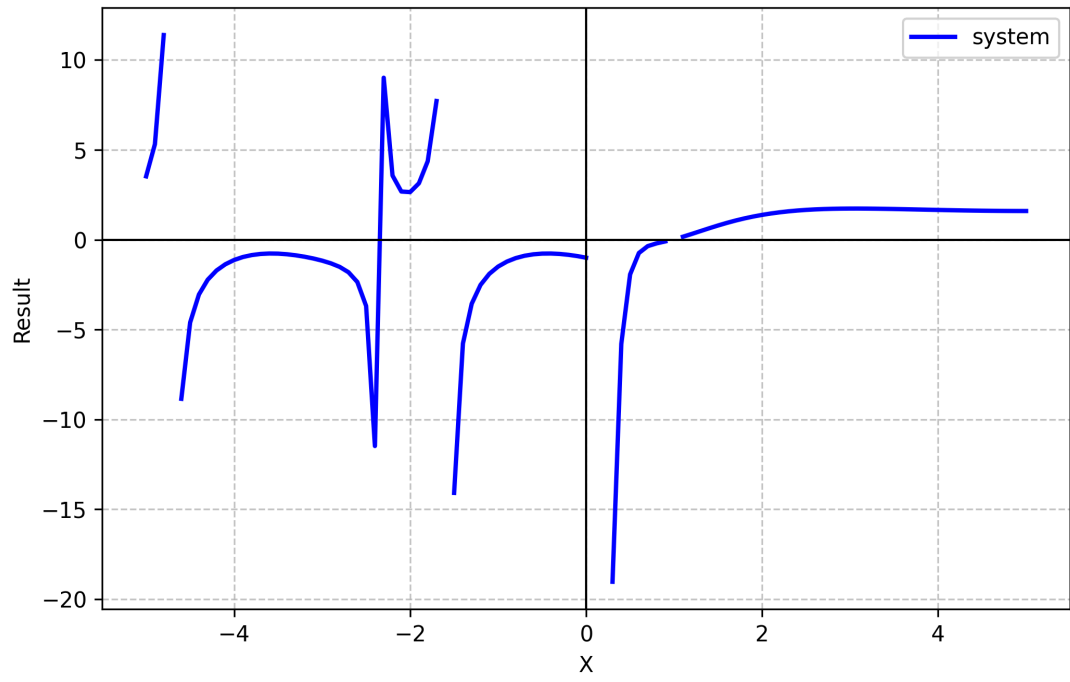


График функции: system



Вывод

В ходе выполнения лабораторной работы были получены практические навыки интеграционного тестирования программного обеспечения. Была успешно разработана модульная система математических функций, базовые компоненты которой реализованы исключительно на основе разложения в ряд Тейлора.

Использование мок-объектов с помощью библиотеки Mockito позволило изолированно протестировать логику работы итоговой системы, исключив влияние погрешностей нижних слоев вычислений. Выгрузка результатов в формат CSV и последующая визуализация данных с помощью графиков наглядно подтвердили правильность математических преобразований программы на всей области допустимых значений. Таким образом, все поставленные задачи были выполнены в полном объеме.